

Лекция 3

kNN, метрики и оценка качества

Лектор

Елизавета Власова

Практики

Кирилл Захаров

Камила Насибуллина

Михаил Подсытник

Игорь Толстокулаков

План лекции

Блок	Что будем делать
1	Повторим стандартизацию и PCA
2	Найдём соседей и построим kNN
3	Разделим данные и выберем гиперпараметры
4	Разберём ошибки и построим ROC / PR
5	Добавим примеры редкого класса с SMOTE
6	Применим поиск соседей в RAG

Стандартизация и PCA

Масштаб признаков, новые оси и проекция

Масштаб признаков и ближайший сосед

У нового клиента Q ищем похожего по возрасту и доходу.

Объект	Возраст, лет	Доход, руб./мес.
Q	30	60 000
A	31	80 000
B	50	61 000

$$d(Q, A) \approx 20\,000, \quad d(Q, B) \approx 1\,000.$$

Вопрос: какой признак почти целиком определяет результат?

Стандартизация: шаг за шагом

На обучающих данных: $\mu = (30, 60\,000)$, $\sigma = (10, 30\,000)$.

$$z_j = \frac{x_j - \mu_j}{\sigma_j}$$

Объект	Возраст в новых единицах	Доход в новых единицах
Q	0	0
A	$(31 - 30)/10 = 0.1$	$(80\,000 - 60\,000)/30\,000 = 0.667$
B	$(50 - 30)/10 = 2$	$(61\,000 - 60\,000)/30\,000 = 0.033$

$$d_z(Q, A) \approx 0.674, \quad d_z(Q, B) \approx 2.000.$$

Теперь ближайший — A . Одна единица по признаку означает одно σ .

Проверка: новый объект

Используем прежние $\mu = (30, 60\,000)$ и $\sigma = (10, 30\,000)$.

Новый клиент: 40 лет, доход 90 000 руб./мес.

- ▶ Какими станут его два признака?
- ▶ Нужно ли пересчитать среднее и стандартное отклонение?
- ▶ Станет ли распределение дохода нормальным после стандартизации?

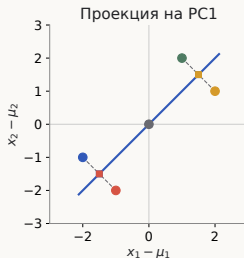
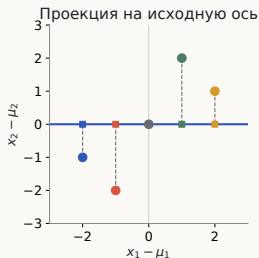
Разбор: преобразование нового объекта

$$z = \left(\frac{40 - 30}{10}, \frac{90\,000 - 60\,000}{30\,000} \right) = (1, 1).$$

- ▶ Параметры преобразования остаются прежними.
- ▶ Стандартизация сдвигает и растягивает шкалу каждого признака.
- ▶ Асимметрия и выбросы сохраняются: нормальное распределение не возникает автоматически.

PCA: выбор направления проекции

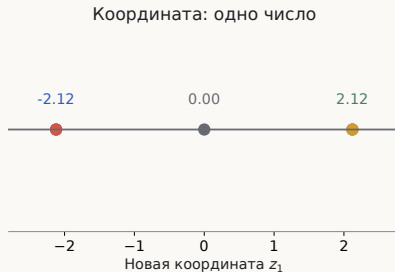
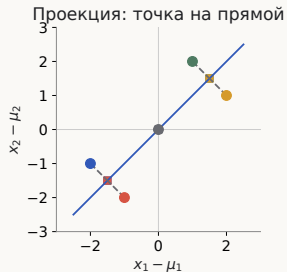
Хотим описать положение объекта одним числом.



Пять объектов, два признака одного масштаба. Среднее уже вычли.

PC1 — единичное направление, на котором проекции имеют наибольшую дисперсию.

PCA: новая координата объекта



$$v_1 = \frac{1}{\sqrt{2}}(1, 1), \quad z_1 = (x - \mu)^T v_1.$$

Точка на прямой задаётся одним числом — расстоянием вдоль новой оси со знаком.

PCA: один расчёт вручную

Среднее исходных признаков $\mu = (3, 3)$. Новый объект $x = (4, 4)$.

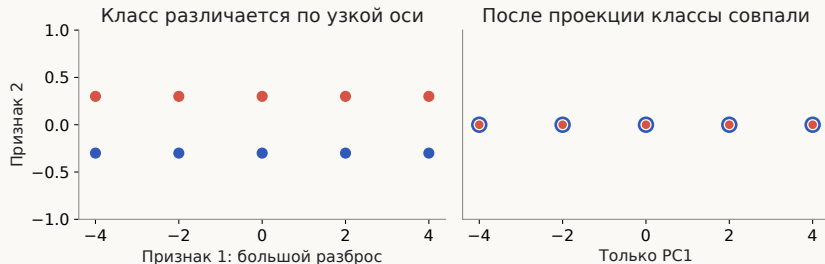
$$x_c = x - \mu = (1, 1), \quad z_1 = \frac{1+1}{\sqrt{2}} \approx 1.414.$$

Откуда взялась ось? Матрица ковариации описывает разброс и совместное изменение признаков:

$$C = \begin{pmatrix} 2.5 & 2 \\ 2 & 2.5 \end{pmatrix}, \quad C v_1 = 4.5 v_1, \quad \frac{4.5}{4.5 + 0.5} = 90\%.$$

Собственный вектор задаёт направление; собственное число — дисперсию проекций.

Потеря информации при PCA



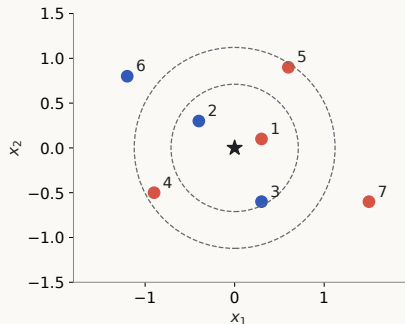
- ▶ Полный ортогональный поворот сохраняет евклидовы расстояния.
- ▶ Отбрасывание компонент меняет расстояния и соседей.
- ▶ Большая сохранённая дисперсия ещё не гарантирует хороший прогноз.

2

kNN и расстояния

Представление объекта определяет его соседей

Класс нового объекта



Звезда — новый объект. Цвет — известный класс обучающих объектов.

Определите класс:

- ▶ при $k = 1$;
- ▶ при $k = 3$;
- ▶ при $k = 5$.

Соседи 1–7 пронумерованы по возрастанию расстояния.

Алгоритм kNN

Сосед	Расстояние	Класс
1	0.316	красный
2	0.500	синий
3	0.671	синий
4	1.030	красный
5	1.082	красный

Для каждого нового объекта:

- ▶ Вычислить расстояния до train.
- ▶ Выбрать k ближайших.
- ▶ Вернуть самый частый класс.

$k = 1$: красный

$k = 3$: синий

$k = 5$: красный

Обучение базового kNN — сохранение примеров (x_i, y_i) ; поиск происходит при предсказании.

ЕВКЛИДОВО И МАНХЭТТЕНСКОЕ РАССТОЯНИЯ

$$d_2(x, u) = \sqrt{\sum_j (x_j - u_j)^2}$$

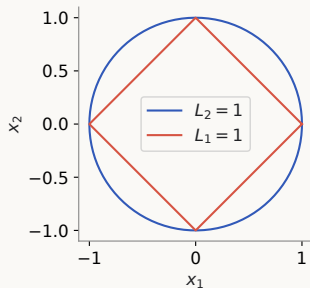
$$d_1(x, u) = \sum_j |x_j - u_j|$$

Для $(0, 0)$ и $(3, 4)$:

$d_2 = 5$, $d_1 = 7$.

На рисунке — все точки на расстоянии 1 от начала координат.

Обобщение: $d_p(x, u) = (\sum_j |x_j - u_j|^p)^{1/p}$, $p \geq 1$.



Косинусная близость

$$\text{sim}_{\cos}(x, u) = \frac{x^T u}{\|x\|_2 \|u\|_2}, \quad d_{\cos} = 1 - \text{sim}_{\cos}.$$

Векторы	Косинусная близость	Что сравниваем
(1, 1) и (2, 2)	1	одинаковое направление
(1, 1) и (1, -1)	0	прямой угол

- ▶ Определена для ненулевых векторов; длина не влияет при положительном масштабировании.
- ▶ После L2-нормирования: $\|\hat{x} - \hat{u}\|_2^2 = 2(1 - \text{sim}_{\cos})$. Ранжирование совпадает.

Выбор расстояния

Представление	Возможный выбор	Что проверить
Числовые признаки	Стандартизация + L_1 или L_2	Масштаб и смысл различий
Векторы текстов	Косинусная близость	Подходит ли модель представления
Бинарные признаки	Хэмминг / Жаккар	Важны ли совместные нули

Строгая метрика: неотрицательность, $d(x, u) = 0 \Leftrightarrow x = u$, симметрия и неравенство треугольника.

Косинусное «расстояние» в общем случае не удовлетворяет всем этим условиям.

Голосование и веса соседей

Для бинарной задачи $y_i \in \{0, 1\}$, $N_k(x)$ — индексы соседей:

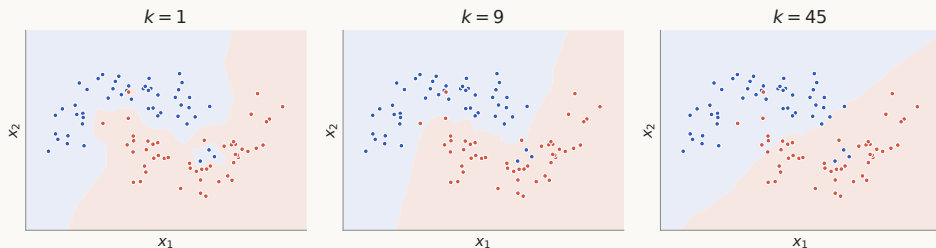
$$s(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i, \quad s_w(x) = \frac{\sum_{i \in N_k(x)} w_i y_i}{\sum_{i \in N_k(x)} w_i}.$$

Три соседа: метки (1, 0, 0)	Без весов	$w_i = 1/d_i$
Расстояния (0.1, 1, 1)	$s = 1/3$	$s_w = 10/12$

Доля голосов задаёт score для положительного класса. Решение зависит от порога.

При нулевом расстоянии требуется отдельное правило; в sklearn голосуют совпавшие объекты.

Число соседей и сложность модели



Малое k : граница чувствительна к отдельным примерам. Большое k : локальные различия сглаживаются.

Bias–variance: слишком сильное сглаживание даёт систематические ошибки; слабое — высокую чувствительность к выборке.

kNN для регрессии

У соседей теперь числовой ответ y_i , например цена квартиры.

$$\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i \quad \text{или} \quad \hat{y}(x) = \frac{\sum_i w_i y_i}{\sum_i w_i}.$$

Три соседа	1	2	3
Цена, млн руб.	8	9	10
Одинаковый вес	1	1	1

Прогноз: 9 млн руб. При неотрицательных весах он лежит между ответами соседей.

Ограничения kNN

Что происходит	Практическое следствие
Храним все обучающие объекты	Память порядка $O(nd)$
Полный перебор на запрос: $O(nd)$	На большой базе нужен индекс
Лишние признаки меняют соседей	Проверяем признаки и уменьшение размерности
В высокой размерности расстояния могут сближаться	Ближайший объект может быть мало похожим

Качество kNN зависит от того, насколько полезно выбранное соседство.

3

Обучение и проверка

Разбиение данных, cross-validation и утечки

Оценка на обучающих данных

У kNN с $k = 1$ каждый обучающий объект может найти самого себя.

Расстояние 0 $\rightarrow$ своя метка $\rightarrow$ accuracy на train равна 1

Это верно для уникальных объектов без конфликтующих меток.

Чтобы измерить обобщение, нужны объекты, которые не участвовали в построении решения.

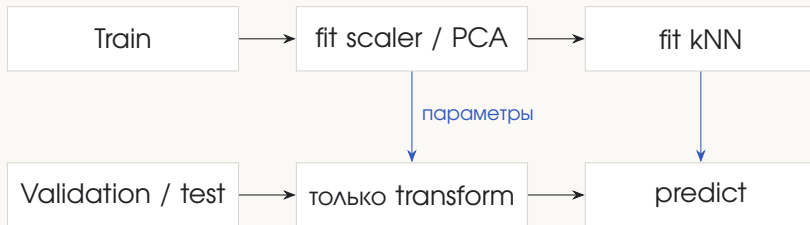
Train, validation и test

Часть данных	Для чего используется
Train	Обучить преобразования и модель
Validation	Выбрать k , расстояние, число компонент, порог
Test	Оценить окончательно выбранное решение

После выбора настроек можно переобучить pipeline на train + validation. Test остаётся независимым.

Пропорции выбираем под размер данных, редкость класса и сценарий применения.

Обучение преобразований



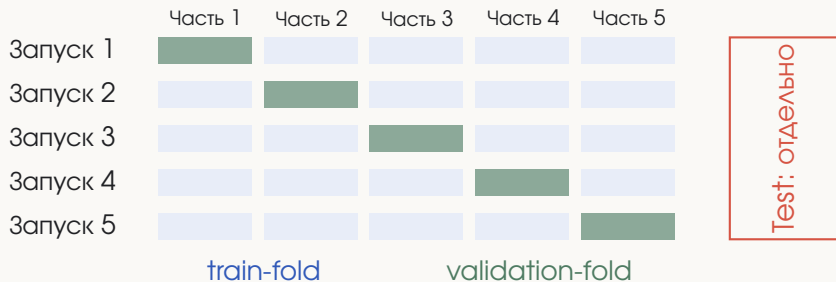
Средние, масштабы и оси PCA определяем по train. На новых данных применяем то же отображение.

Разбиение под сценарий применения

Новые объекты	Как делить	Типичная ошибка
Независимые объекты	Случайно / со стратификацией	Дубликаты по обе стороны
Новые пользователи	По ID пользователя	Один пользователь в train и test
Будущие события	По времени	Будущие события попали в train

Стратификация сохраняет доли классов, но не устраняет зависимость между объектами.

K-fold cross-validation



Для каждой настройки — пять обучений и пять оценок. Сравниваем среднее и разброс.

В каждом запуске scaler, PCA и модель обучаются заново на train-fold.

Подбор гиперпараметров

Пример пространства поиска: $k \in \{3, 9, 21\}$, расстояние L_1/L_2 , веса одинаковые / по расстоянию.

$$3 \times 2 \times 2 = 12 \text{ настроек}, \quad 12 \times 5 = 60 \text{ обучений при 5-fold CV.}$$

- ▶ До сравнения задаём основную метрику и допустимые ограничения.
- ▶ Выбираем настройку по validation или CV.
- ▶ Фиксируем выбор и только затем оцениваем на test.

Проверка: где утечка данных?

Студент сделал так:

1. Обучил StandardScaler на всех объектах.
2. Применил PCA ко всей таблице.
3. Разделил данные на train и test.
4. Сравнил 20 значений k по test accuracy.
5. Опубликовал лучший результат.

Что нужно изменить и в каком порядке?

Разбор: порядок эксперимента

1. Отделить `test` до обучения преобразований.
2. На оставшихся данных организовать `validation` или `CV`.
3. Внутри каждой обучающей части обучать `scaler` → `PCA` → `kNN`.
4. Выбрать k и остальные настройки по `validation` / `CV`.
5. После фиксации решения получить одну итоговую оценку на `test`.

Pipeline помогает удерживать этот порядок внутри `cross-validation`.

4

Метрики классификации

Ошибки, порог решения и кривые ROC / PR

Матрица ошибок

Положительный класс: нежелательное письмо. Рассмотрим 100 писем.

Истинный класс	Предсказали «спам»	Предсказали «обычное»
Спам: 5	TP = 3	FN = 2
Обычное: 95	FP = 2	TN = 93

- ▶ FP: обычное письмо ошибочно попало в спам.
- ▶ FN: спам остался во входящих.

Accuracy при дисбалансе классов

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{3 + 93}{100} = 96\%.$$

Если всегда отвечать «обычное письмо», accuracy уже равна 95%.

Из пяти спам-писем модель нашла только три

Accuracy считает все верные ответы вместе. На редком классе нужен отдельный разбор ошибок.

Precision, recall и F1

Для той же матрицы: $TP = 3$, $FP = 2$, $FN = 2$.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{3}{5} = 0.6$$

Среди писем, отправленных в спам, 60% действительно спам.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{3}{5} = 0.6$$

Среди всех спам-писем мы нашли 60%.

$$F_1 = \frac{2PR}{P + R} = \frac{2TP}{2TP + FP + FN} = 0.6.$$

Score и порог решения

В новом учебном примере восемь объектов. Score выше — положительный класс вероятнее.

Объект	A	B	C	D	E	F	G	H
Score	.95	.85	.80	.70	.60	.40	.30	.10
Истина y	1	0	1	1	0	0	1	0

$$\hat{y} = \mathbf{1}[s(x) \geq t].$$

При $t = 0.70$ положительными считаем A, B, C, D.

Посчитайте: TP , FP , FN , TN .

Изменение порога

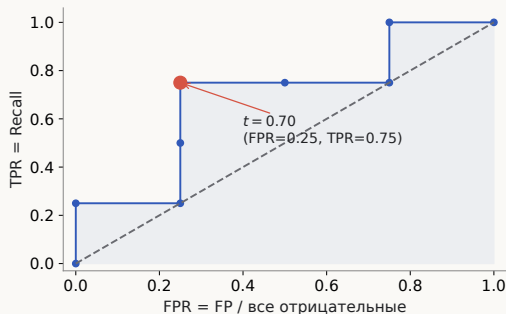
t	TP	FP	FN	TN	Precision	Recall	FPR
.90	1	0	3	4	1.00	.25	.00
.70	3	1	1	3	.75	.75	.25
.30	4	3	0	1	.57	1.00	.75

$$\text{FPR} = \frac{FP}{FP + TN}$$

Понижаем порог: положительных решений становится больше; TP и FP не убывают.

Recall не убывает. Precision может меняться немонотонно.

ROC-кривая



По горизонтали:

доля отрицательных, ошибочно признанных положительными.

По вертикали:

доля найденных положительных.

Каждая точка соответствует порогу.

Лучше — ближе к левому верхнему углу.

ROC-AUC: качество ранжирования

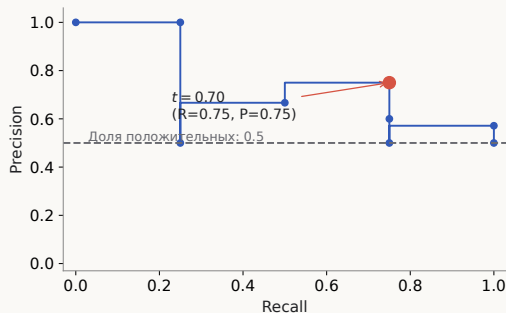
ROC-AUC — площадь под ROC-кривой. В нашем примере:

$$\text{AUC} = \frac{11}{16} = 0.6875.$$

Из 16 пар «положительный — отрицательный» в 11 score положительного выше.

- ▶ Диапазон AUC: $[0, 1]$; случайный порядок в среднем даёт 0.5.
- ▶ При совпавших score пара даёт половину балла.
- ▶ Одна площадь не определяет лучший рабочий порог.

PR-кривая и её площадь



По горизонтали: recall.

По вертикали: precision.

Удобна для анализа качества на положительном классе.

Average precision (AP):

$AP \approx 0.747$,

площадь трапециями ≈ 0.707 .

AP и трапецидальная PR-AUC — разные способы суммирования.

Редкий класс: ROC и PR

Пусть при фиксированном пороге $TPR = 80\%$, $FPR = 10\%$. Сравним по 10 000 объектов.

Доля положительных	TP	FP	Recall	Precision
50%	4 000	500	80%	88.9%
1%	80	990	80%	7.5%

Точка ROC одинакова, а состав положительных решений резко отличается.

PR зависит от доли положительного класса. Сравниваем модели на одной целевой выборке.

Проверка: выбор рабочего порога

Автоматически удаляем спам. По условию задачи precision должен быть не ниже 90%.

Порог на validation	Precision	Recall
t_1	.96	.35
t_2	.91	.70
t_3	.78	.88

Какой порог выбрать, если среди допустимых решений хотим найти больше спама? Где проверить результат после выбора?

Разбор: метрика и ограничение

Выбираем t_2 : precision = 91%, recall = 70%.

- ▶ t_3 нарушает ограничение.
- ▶ t_1 допустим, но находит меньше спама.
- ▶ После фиксации порога оцениваем решение на test.

Для модерации с ручной проверкой ограничением может стать число проверок в день. Тогда критерий выбора будет другим.

5

Дисбаланс и SMOTE

Генерация примеров редкого класса

Работа с дисбалансом классов

Приём	Что меняет	Ограничение
Изменить порог	Правило принятия решения	Не улучшает ранжирование
Random oversampling	Повторяет редкие объекты	Новых наблюдений не появляется
Undersampling	Удаляет часть частого класса	Можно потерять полезные примеры
SMOTE	Добавляет точки между соседями	Предполагает допустимость интерполяции

Все варианты сравниваем на одной исходной validation-выборке.

SMOTE: синтетический объект



Для x_i случайно выбираем x_j среди k_{SMOTE} ближайших объектов того же класса.

$$x_{new} = x_i + \alpha(x_j - x_i), \quad \alpha \sim U(0, 1).$$

При $x_i = (1, 1)$, $x_j = (5, 3)$, $\alpha = 0.25$:

$$x_{new} = (2, 1.5).$$

Синтетическая точка наследует метку редкого класса.

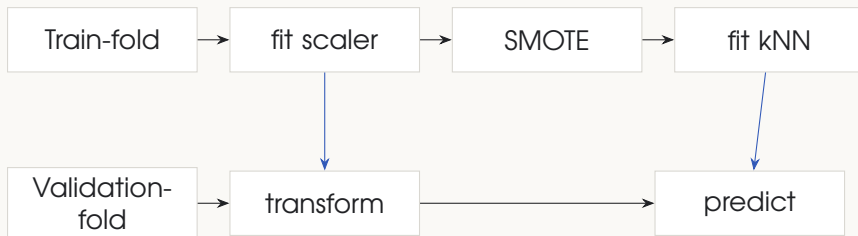
Ограничения SMOTE



- ▶ Отрезок может пересечь область другого класса.
- ▶ Точки около выбросов усиливают ошибочную структуру.
- ▶ Обычный SMOTE предполагает числовые признаки и осмысленную интерполяцию.

SMOTE не добавляет независимых наблюдений и не гарантирует улучшения качества.

SMOTE внутри обучающего фолда



Validation и test сохраняют исходные объекты и доли классов.

Реализация: `imblearn.pipeline.Pipeline`. На predict шаг ресемплинга пропускается.

6

Поиск соседей и RAG

Тот же поиск, другая операция над
результатом

Поиск и классификация

Задача	Что находим	Что возвращаем
kNN-классификация	Соседей в train	Класс по голосам
kNN-регрессия	Соседей в train	Среднее значений
Векторный поиск	Соседей в корпусе	Сами документы или фрагменты

Для retrieval метки классов не обязательны: нужен вектор запроса и сопоставимые векторы документов.

Текст как вектор

Учебный словарь из трёх слов: «данные», «масштаб», «соседи». Числа — количество вхождений.

Текст	Вектор
Q : «Данные и масштаб»	(1, 1, 0)
D_1 : «Данные: масштаб. Данные: масштаб»	(2, 2, 0)
D_2 : «Данные и соседи. Соседи»	(1, 0, 2)
D_3 : «Соседи»	(0, 0, 1)

Для настоящего корпуса можно использовать TF-IDF или готовую модель эмбедингов. Представления запроса и документов должны быть совместимы.

Ранжирование по близости

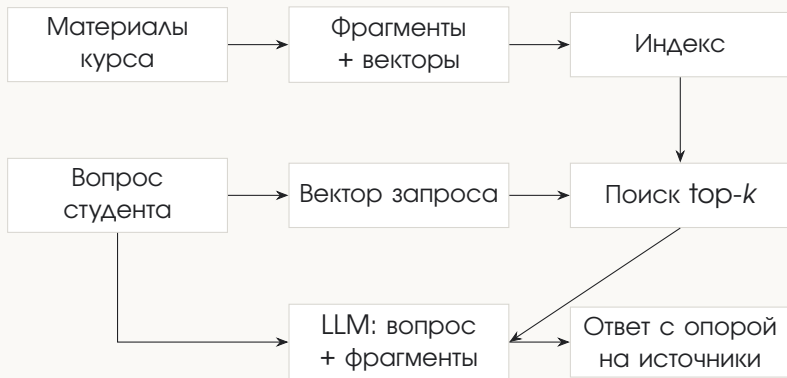
Для запроса $Q = (1, 1, 0)$ сравним документы из предыдущего слайда.

Документ	Косинусная близость	Место
$D_1 = (2, 2, 0)$	$4/(\sqrt{2}\sqrt{8}) = 1$	1
$D_2 = (1, 0, 2)$	$1/(\sqrt{2}\sqrt{5}) \approx 0.316$	2
$D_3 = (0, 0, 1)$	0	3

Top-2: D_1, D_2 . Здесь сходство связано с общими словами.

StandardScaler меняет геометрию эмбедингов. Для косинусного поиска обычно нормируют длину вектора, если это соответствует модели.

RAG: поиск перед генерацией



Retrieval-augmented generation: найденный контекст передаётся модели при формировании ответа.

Пример: ответ по материалам курса

Вопрос: «Нужно ли обучать scaler заново для тестовых данных?»

ID	Найденный текст
D_A	«Среднее и масштаб оценивают по train. Для test используют transform с сохранёнными параметрами».
D_B	«При cross-validation преобразования обучают заново внутри каждого train-fold».

Ответ с опорой на контекст: «Нет. Примените параметры, оценённые на train (D_A). В CV отдельный fit нужен для каждого обучающего фолда (D_B)».

Точный и приближённый поиск

Способ	Что делает	Компромисс
Полный перебор	Считает расстояния до всех векторов	Точный top-k, высокая стоимость
Индекс ANN	Просматривает часть кандидатов	Скорость, память, возможные пропуски

ANN = approximate nearest neighbors; примеры индексов — HNSW и IVF.

Проверяем задержку и долю найденных соседей относительно точного поиска. Это ещё не оценка смысловой релевантности.

Оценка поиска и ответа

Отдельный пример: пять документов. Для запроса релевантны $R = \{D_1, D_3, D_5\}$.
Найдено $S_2 = \{D_1, D_2\}$.

$$\text{Precision@2} = \frac{|R \cap S_2|}{2} = \frac{1}{2}, \quad \text{Recall@2} = \frac{|R \cap S_2|}{|R|} = \frac{1}{3}.$$

- ▶ Поиск: нашёл ли нужные фрагменты и за какое время?
- ▶ Ответ: корректен ли он, подтверждается ли найденными источниками?
- ▶ Настройки retrieval выбираем на validation-запросах; test-запросы оставляем для итоговой оценки.

Проверка: эксперимент с RAG

Есть материалы курса и 100 вопросов с отмеченными релевантными фрагментами.

- ▶ По всем 100 вопросам выбрали модель эмбедингов и k .
- ▶ На этих же вопросах посчитали $\text{Recall}@k$.
- ▶ По высокому $\text{Recall}@k$ заключили, что ответы LLM правильные.

Где ошибки оценивания? Как исправить эксперимент?

Разбор: проверка RAG

- ▶ Выбор настроек и итоговая оценка выполнены на одних вопросах: нужен отдельный test-набор запросов.
- ▶ Recall поиска оценивает найденные фрагменты. Ответ LLM нужно проверить отдельно на корректность и подтверждённость источниками.

Общая схема сохраняется: представление → поиск → решение → независимая оценка.

Теоретический минимум

Тема	Нужно уметь объяснить
Масштаб и PCA	Масштаб по train; проекция на новые оси; потеря компонент меняет соседство
kNN и расстояния	L_1 , L_2 , cosine; выбор k ; голоса, веса и регрессия
Оценивание	Train / validation / test; CV; fit всех преобразований внутри train-fold
Классификация	TP/FP/FN/TN; accuracy, precision, recall, F1; score и порог; ROC/PR и AUC
SMOTE	Интерполяция соседей одного класса; только train
Поиск и RAG	Top- k фрагментов → контекст LLM; поиск и ответ оцениваем отдельно

Ко(т)нец презентации



Сегодня мы собрали полный эксперимент с методом ближайших соседей.

Вопросы?

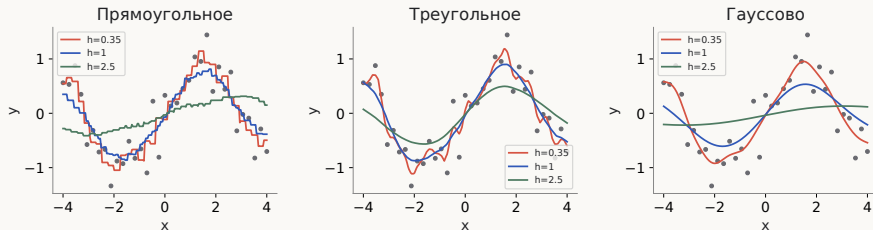
Материалы для самостоятельной работы

- ▶ scikit-learn: Importance of Feature Scaling
- ▶ scikit-learn: Nearest Neighbors
- ▶ scikit-learn: Cross-validation
- ▶ scikit-learn: Metrics and scoring
- ▶ imbalanced-learn: Over-sampling и SMOTE
- ▶ imbalanced-learn: Data leakage
- ▶ Lewis et al. (2020): Retrieval-Augmented Generation
- ▶ Faiss: MetricType and distances

Приложение: ядро и ширина окна

Вес по расстоянию можно задать через ядро K и ширину окна $h > 0$:

$$w_i = K\left(\frac{d(x, x_i)}{h}\right), \quad \hat{y}(x) = \frac{\sum_i w_i y_i}{\sum_i w_i}.$$



Суммируем по всем train-объектам; требуется ненулевая сумма весов.

Приложение: метрики регрессии

$$\text{MAE} = \frac{1}{n} \sum_i |y_i - \hat{y}_i|, \quad \text{MSE} = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2, \quad \text{RMSE} = \sqrt{\text{MSE}}.$$

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}.$$

- ▶ MAE и RMSE имеют единицы целевой переменной; MSE сильнее выделяет большие ошибки.
- ▶ R^2 может быть отрицательным; при ненулевой дисперсии у верхняя граница равна 1.
- ▶ В знаменателе R^2 — среднее по оцениваемой выборке. Для константного y нужно отдельное соглашение.